Aula prática: explorando `vi.fn()` no REPL do Node

Bloco 0 — entrar no laboratório

Rode isso

```
node
```

Depois, no REPL:

```
const { vi } = await import('vitest')
```

Observe isso

Você não deve receber erro.

Conclusão

Agora você está com o `vi` carregado dentro do REPL e pode explorar mocks manualmente.

Bloco 1 — criar o mock mais simples possível

Rode isso

```
const fn = vi.fn()
```

Depois:

```
typeof fn
fn
fn.mock
```

Observe isso

- `typeof fn` deve ser `'function'`
- `fn` é chamável
- `fn.mock` é o objeto que guarda o estado interno do mock

Conclusão

Um mock do Vitest é uma **função com memória**.

Bloco 2 — chamar o mock e inspecionar argumentos

Rode isso

```
fn('a')
fn('b', 123)
fn({ id: 1 })
```

Depois:

```
fn.mock.calls
```

Observe isso

Você verá algo conceitualmente assim:

```
[
  ['a'],
  ['b', 123],
  [{ id: 1 }]
]
```

Conclusão

`mock.calls` registra **todas as chamadas**, na ordem em que aconteceram.

Cada posição representa uma execução, e o conteúdo é a lista de argumentos daquela execução.

Bloco 3 — navegar pelo histórico de chamadas

Rode isso

```
fn.mock.calls.length
fn.mock.calls[0]
fn.mock.calls[1]
fn.mock.calls[2]
fn.mock.calls.at(-1)
```

Observe isso

- `.length` mostra quantas vezes a função foi chamada
- `calls[0]` mostra os argumentos da primeira chamada
- `calls.at(-1)` mostra a última chamada

Conclusão

Você pode tratar `mock.calls` como um log estruturado.

Bloco 4 — entender o retorno padrão

Rode isso

```
fn.mock.results
```

Observe isso

Como o mock não tem implementação, o retorno padrão é `undefined` .

Algo como:

```
[
  { type: 'return', value: undefined },
  { type: 'return', value: undefined },
  { type: 'return', value: undefined }
]
```

Conclusão

Além dos argumentos, o mock também registra o que aconteceu em cada chamada.

Bloco 5 — cruzar chamada e resultado

Rode isso

```
fn.mock.calls[1]
fn.mock.results[1]
```

Observe isso

Você está olhando para a **mesma execução**, mas sob dois ângulos:

- com quais argumentos foi chamada
- o que ela retornou

Conclusão

Pense assim:

- `calls[n]` = entrada
- `results[n]` = saída

Bloco 6 — criar um mock com comportamento real

Rode isso

```
const upper = vi.fn((s) => s.toUpperCase())
```

Depois:

```
upper('john')
upper('mary')
```

E então:

```
upper.mock.calls
upper.mock.results
```

Observe isso

Você verá algo como:

```
upper.mock.calls
// [ ['john'], ['mary'] ]

upper.mock.results
// [
//   { type: 'return', value: 'JOHN' },
//   { type: 'return', value: 'MARY' }
// ]
```

Conclusão

Quando existe implementação, `results` passa a registrar o retorno real.

Bloco 7 — trocar comportamento com `mockImplementation`

Rode isso

```
upper.mockImplementation((s) => `mocked:${s}`)
```

Depois:

```
upper('alice')
upper.mock.calls
upper.mock.results
```

Observe isso

A nova chamada já usa a nova implementação.

Conclusão

`mockImplementation` muda o comportamento do mock e continua acumulando histórico.

Bloco 8 — diferença entre valor fixo e implementação

Rode isso

```
const fixed = vi.fn()
fixed.mockReturnValue('constant')
```

Depois:

```
fixed('x')
fixed('y')
fixed.mock.calls
fixed.mock.results
```

Observe isso

Os argumentos mudam, mas o retorno será sempre o mesmo.

Conclusão

`mockReturnValue` é para **retorno fixo**.

Quando o retorno depende dos argumentos, use `mockImplementation`.

Bloco 9 — sequência com `mockReturnValueOnce`

Rode isso

```
const seq = vi
  .fn()
  .mockReturnValue('default')
  .mockReturnValueOnce('first')
  .mockReturnValueOnce('second')
```

Depois:

```
seq()
seq()
seq()
seq()
```

E então:

```
seq.mock.results
```

Observe isso

A sequência esperada é:

- primeira chamada → `'first'`
- segunda → `'second'`
- terceira → `'default'`
- quarta → `'default'`

Conclusão

Os métodos `Once` formam uma fila de comportamentos temporários.

Bloco 10 — comportamento baseado em argumento, uma vez só

Rode isso

```
const sample = vi.fn((s) => s.toUpperCase())
sample.mockImplementationOnce((s) => `first:${s}`)
```

Depois:

```
sample('john')
sample('mary')
sample.mock.results
```

Observe isso

- primeira chamada usa a implementação temporária
- segunda chamada volta para a implementação padrão

Conclusão

`mockImplementationOnce` é o caminho quando a chamada especial depende dos parâmetros.

Bloco 11 — quando o mock lança erro

Rode isso

```
const risky = vi.fn((value) => {
  if (value === 'boom') {
    throw new Error('failure')
  }

  return `ok:${value}`
})
```

Depois:

```
risky('a')
```

Agora rode:

```
try {
  risky('boom')
} catch (error) {
  error.message
}
```

E então:

```
risky.mock.calls
risky.mock.results
```

Observe isso

Você deve ver algo como:

- uma entrada com `type: 'return'`
- uma entrada com `type: 'throw'`

Conclusão

`mock.results` não registra só retorno. Ele também registra exceções lançadas.

Bloco 12 — limpar histórico sem perder comportamento

Rode isso

```
const clearable = vi.fn().mockReturnValue(10)

clearable('a')
clearable('b')
```

Inspecione:

```
clearable.mock.calls
clearable.mock.results
```

Agora:

```
clearable.mockClear()
```

Depois:

```
clearable.mock.calls
clearable.mock.results
clearable()
```

Observe isso

- `calls` e `results` ficam vazios
- o mock continua retornando `10`

Conclusão

`mockClear()` limpa o histórico, mas preserva a implementação.

Bloco 13 — resetar histórico e implementação

Rode isso

```
const resettable = vi.fn(() => 1)
resettable()
resettable.mockImplementation(() => 2)
resettable()
```

Inspecione:

```
resettable.mock.results
```

Agora:

```
resettable.mockReset()
```

Depois:

```
resettable()
resettable.mock.results
```

Observe isso

No Vitest, como o mock nasceu com implementação original `() => 1`, após `mockReset()` ele volta para essa implementação original.

Conclusão

Esse é um ponto importante para quem vem do Jest: o reset do Vitest pode restaurar a implementação original do mock.

Bloco 14 — nomear o mock

Rode isso

```
const named = vi.fn().mockName('findUserById')
named.getMockName()
```

Observe isso

O nome retornado será `'findUserById'`.

Conclusão

Dar nome ao mock ajuda bastante na legibilidade e no debugging.

Bloco 15 — explorar async com `mockResolvedValue`

Rode isso

```
const repo = vi.fn().mockResolvedValue({ id: 'u1' })
const promise = repo('123')
```

Depois:

```
repo.mock.calls
repo.mock.results
repo.mock.settledResults
```

Agora:

```
await promise
repo.mock.settledResults
```

Observe isso

- `calls` registra os argumentos
- `results` registra que a chamada retornou uma Promise
- `settledResults` mostra o estado da Promise
- antes do `await`, pode aparecer como incompleta
- depois do `await`, aparece como resolvida

Conclusão

Para mocks async, `settledResults` é a visão mais fiel do desfecho da Promise.

Bloco 16 — rejeição async com `mockRejectedValue`

Rode isso

```
const failingRepo = vi.fn().mockRejectedValue(new Error('db error'))
```

Depois:

```
try {
  await failingRepo('123')
} catch (error) {
  error.message
}
```

Agora:

```
failingRepo.mock.calls
failingRepo.mock.results
failingRepo.mock.settledResults
```

Observe isso

Você terá o registro da chamada, o retorno da Promise e o desfecho rejeitado.

Conclusão

Em cenário backend, isso é muito útil para simular falhas de repositório, gateway, client HTTP, fila, etc.

Exercício guiado final

Agora rode este bloco inteiro, linha por linha:

```
const demo = vi.fn((s) => s.toUpperCase())

demo('a')
demo('b')

demo.mock.calls
demo.mock.results

demo.mockImplementationOnce((s) => `once:${s}`)
demo('c')
demo('d')

demo.mock.calls
demo.mock.results

demo.mockClear()
demo.mock.calls
demo.mock.results

demo('z')
demo.mock.calls
demo.mock.results
```

Observe isso

Você vai conseguir enxergar, em uma mini sessão só:

- histórico acumulado
- troca temporária de implementação
- limpeza de histórico
- reutilização do mesmo mock após limpar

Conclusão

Esse exercício já te dá a intuição central do objeto de mock do Vitest.

Mapa mental para decorar

```
fn.mock.calls
fn.mock.results
fn.mock.instances
fn.mock.settledResults
```

e:

```
fn.mockClear()
fn.mockReset()
fn.mockImplementation()
fn.mockImplementationOnce()
fn.mockReturnValue()
fn.mockReturnValueOnce()
fn.mockResolvedValue()
fn.mockRejectedValue()
fn.mockName()
```

Fechamento da aula

Se você sair dessa exploração lembrando só isto, já está no caminho certo:

- `calls` = como foi chamado
- `results` = o que retornou ou lançou
- `settledResults` = como a Promise terminou
- `mockReturnValue` = valor fixo
- `mockImplementation` = retorno dependente dos argumentos
- `mockClear` = limpa histórico
- `mockReset` = limpa e reseta comportamento